

C FINAL TEST

IMPERIAL COLLEGE LONDON

DEPARTMENT OF COMPUTING

Regexes and Finite State Machines

Monday 9th June 2025
THREE HOURS

- This final test is **assessed**. Part 1 carries 10% of the marks, part 2 carries 40% of the marks, and part 3 carries 50% of the marks. Attempt all parts. Each part may be done entirely independently.
- Your code must compile correctly. Comment out code that does not compile before you finish. Marks will be deducted if your code doesn't compile.
- In addition, **up to half the marks** for any one question will be awarded for your code **actually working**. We'll attempt to give some credit for code that had the right idea but contained some errors, but not more than half. This may appear to be a harsher marking criterion than used for Haskell, Kotlin and Java earlier in the year.
- **Important:** only modify the files each task tells you to. All other files should be considered read-only. All files for each question are in a separate `~/re/PartN` directory.
- Feel free to define any of your own helper functions which would make your code more elegant; make them static. Remember header files such as `<string.h>` and `<assert.h>`. Remember man pages (`man strcpy` etc).
- Use any editor you like, no assistance will be given for editors. Compile your code however you like; we recommend using **CBuild (aka cb)** as usual, and have supplied suitable `.cbuild` files for each question. No assistance will be given with compiling your code any other way.

Introduction - Problem Description

We are continuing the Practice test's concentration on **Regular expressions** or **regexes**. In the Practice test, we defined a simple version of regexes and proceeded to build a recursive **match regex to string** function.

In this test, we are going to *keep using exactly the same version of regexes*, so that we can reuse most of the code we saw last time.

But now we're going to build an alternative – far more efficient – method of matching a regex to a string, using **Finite State Machines**.

To get started, we will do a little string manipulation work on a module that checks that a regex has been simplified correctly. Then we will do some work on completing a **Finite State Machine** framework that can be used to do regex matching if we can build a suitable FSM. Then, I will outline a method for building a FSM for any of our simplified regexes, and we will complete several stubbed functions in the module that turns a regex into a FSM.

If everything works, and as a bonus, you may like to assemble all the bits together and see whether or not you have successfully implemented **searchstr**.

Recap - Simple Regular Expressions

(This material is **identical** to this section in the practice test).

We will work with a small subset of full-blown regular expressions, which offer a good proportion of the power of full-blown regexes while being much simpler to implement. Specifically:

Single Character Regexes

- A *terminal character* such as **a** matches itself.
- A *range* such as **[abc]** matches any one of those characters, i.e. either an **'a'** or a **'b'** or a **'c'**. That is, it represents a set from which we match any member.

A range can contain any number of characters, and for convenience you can write **[a-j]** as a shorthand for **[abcdefghij]**. A range may contain a mixture of individual characters and contiguous ranges, such as **[aep-tz]**.

- An *any* (written as **.**) matches *any single non-NUL character* - i.e. it matches any single character apart from the end of the target string.

Sequencing and anchoring

- A *sequence* of any two or more simple patterns placed next to each other, such as **ab[c-d]** matches each simple pattern in turn against adjacent characters in the target string. In this case, it matches any target string that contains, somewhere within it, an **'a'** immediately followed by a **'b'** immediately followed by either a **'c'** or a **'d'**.

- Note that any string literal regex such as `hello` (as long as it doesn't contain any special regex characters) matches that literal string, somewhere within the target string.
- Normally a regex matches anywhere in the target string. A *start anchor* (written as `^` at the start of a regex) causes that regex to only match at the start of the target string. So for example `^h[ei]` only matches target strings in which the first character is an 'h' and the second character is either 'e' or 'i' – that is, any target string that starts with `he` or `hi`.
- Similarly, an *end anchor* (written as `$` at the end of a regex) causes the regex to only match against the end of the target string. So for example `h[ei]$` only matches target strings in which the last two characters are 'h' and either 'e' or 'i' – that is, any target string ending in `he` or `hi`.

Repetitions

- *Optional: match zero or one time* – any single character pattern may be made *optional*, by appending a `?` to it. So, for example, `a?` matches the empty string or a single 'a', `[a-c]?` matches the empty string or a single 'a', 'b' or 'c', and `.?` matches the empty string or any single non-empty character (that means it always matches, even at the end of the target string).
- *Kleene Star Repetition: match zero or more times* – any single character pattern may be repeated *zero or more times*, by appending a `*` to it. So, for example, `a*` matches the empty string, or 'a' or 'aa', or 'aaa'... or any number of adjacent 'a's, such as 'aaaaaaaaaaaaaaaaaaaaa'.
`[a-c]*` matches the empty string, or a single 'a', 'b' or 'c', or two adjacent letters each from the set 'a'.. 'c' (eg. `bc`), or any number of adjacent letters from the set 'a'.. 'c' (eg. `abcaababcaabbcacb`).
`.*` matches the empty string or any single character or any two adjacent characters.. or any number of adjacent characters. In other words it matches any amount of text. This is often used as a separator, as in `[a-z].*[0-9]` which matches a letter followed later by a digit.
- *Kleene Plus Repetition: match one or more times* – any single character pattern may be repeated *one or more times*, by appending a `+` to it. So, for example, `a+` matches 'a' or 'aa' or 'aaa'... or any number of adjacent 'a's, such as 'aaaaaaaaaaaaaaaaaaaaa'. But unlike `a*` it doesn't match the empty string.
`[a-c]+` matches a single 'a', 'b' or 'c', or two adjacent letters each independently chosen from the set 'a'.. 'c' (eg. `bc`), or any number of adjacent letters from the set 'a'.. 'c' (eg. `abcaababcaabbcacb`). But unlike `[a-c]*` it doesn't match the empty string.
`.+` matches any single character or any two adjacent characters.. or any number of adjacent characters. In other words it matches any non-empty amount of text. Unlike `.*` it doesn't match the empty string.

Pre-supplied files

You are supplied with a number of files in the `~/re` directory:

<code>README:</code>	readme file that describes all the other files
<code>labsearchstr:</code>	pre-compiled working regex string matcher
<code>summarisetests:</code>	C-Tools style test summariser
<code>Part1/</code>	material for Part 1 (described later)
<code>Part2/</code>	material for Part 2 (described later)
<code>Part3/</code>	material for Part 3 (described later)
<code>lib/</code>	a library directory containing header files and all pre-compiled sample solution code.

As in the practice tests, you can use `labsearchstr` to explore your understanding of regex pattern matching against particular target strings. The **README** file gives examples of this.

Part 1 : Getting Started (10%)

You are supplied with a number of files in the `~/re/Part1` directory:

<code>README:</code>	readme file that describes Part 1
<code>checkregex.[ch]:</code>	Module to check a regex for banned combinations - skeleton
<code>testcheck.c:</code>	Unit test program for checkregex module
<code>.cbuild:</code>	Custom CB config file to compile unit test(s)

Specific Tasks in Part 1

1. Cd into the **Part1/** directory, read the **README** file, and then compile everything up via `cb`. Run the unit test `./testcheck` and you will see that it fails some of it's tests. Why? The file **checkregex.c** contains a number of checking functions that check that specific combinations of regex atoms do not occur (in fact, an earlier regex simplification phase goes through and replaces many of these banned combinations with simpler ones – this simplification phase need not concern you).
2. However, one of those checking functions is a mere stub:

```
// char dup = char_in_common( str1, str2 );  
//      Return a duplicate char found in the two strings str1 and str2  
//      (if they have one or more characters in common, i.e. considered  
//      as sets they intersect, or '\0' if they have no char in common.
```

3. Implement this stubbed function, in order to get the unit test to pass all it's tests. Note that all the `.cbuild` files have a `cb --test` target that invokes `summarisetests` on the appropriate unit test programs (here `./testcheck`) for you.

4. Also remember that the post-processing that `summarisetests` does is confusing if your code dies with a seg fault or trips an assertion – in this case it is better if you simply run `./testcheck` manually. When your test program appears to terminate normally, you can run it via `cb --test` to see the useful tests passed/tests failed summary.
5. When all your unit tests pass, I recommend that you also run `valgrind ./testcheck` to check that it doesn't leak memory, or overrun the ends of arrays. Marks will be deducted for memory leaks. I won't mention `valgrind` again, but you should run it on each program you generate once you believe it works correctly.
6. **Hint:** You are welcome to implement `char_in_common()` however you like, one possible option is using `strchr()` in a loop. Reread `man strchr` to refresh your memory of how it works, and how to use it. Or you can use nested loops if you prefer.

Part 2 : Finite State Machines (40%)

You are supplied with a number of files in the `~/re/Part2` directory:

<code>README:</code>	<code>readme file that describes Part 2</code>
<code>fsm.[ch]:</code>	<code>Finite State Machine module - fsm.c is a skeleton.</code>
<code>token.[ch]:</code>	<code>Tokens and token acceptor function.</code>
<code>testfsm.c:</code>	<code>Unit test program for fsm module.</code>
<code>.cbuild:</code>	<code>Custom CB config file to compile unit tests.</code>

First, let's familiarise ourselves with the FSM module, to see how FSMs may be represented in C, explaining what you need to know about FSMs at the same time.

Cd into the **Part2/** directory, read the **README** file, compile everything up via `cb`, then read on.

Introduction to Finite State Machines

Finite State Machines – or **FSMs** – are a type of labelled state transition system with a finite number of states. We can consider them in two different ways: in **static fashion** and in **dynamic fashion**.

First, in a **static fashion**, a **FSM** consists of `N` numbered **states** (`0..N-1`) and any number of **labelled transitions** (let's call them **arcs**) between states. There is a **start state** – this is always state 0. There is a **success state** – this is always state `N`. In addition, there is a **failure** state which causes the FSM matching to terminate immediately: `fsm.h` contains `#define FAIL -1`.

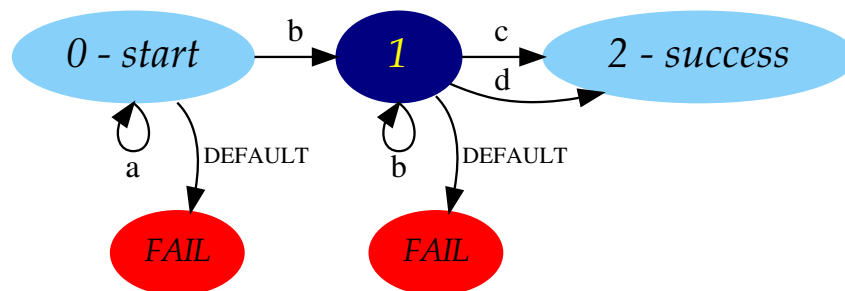
Second, in a **dynamic fashion**, we create a fleet of one or more instances of the static FSM rules and run them against a target string. Each instance (or **active FSM**, or **ASM** for short) has its own private state.

FSMs can be used for many purposes, but we are going to concentrate on the obvious application: any regex can be described by a suitable **FSM**, and pattern matching of that regex against a target string can be efficiently performed by running a fleet of ASMs against successive characters of the target string. In this application, each arc label is an integer **token**, either a single character or a special token value such as **ANY** or **DEFAULT**.

Let's see an example. The regex `^a*b+[cd]` matches any string comprising the following at the start of the string:

- Zero or more 'a's, followed by
- One or more 'b's, followed by
- Either a 'c' or a 'd', followed by
- zero or more additional characters that we don't care about.

This can be turned into the following **FSM**:



(We will learn how to turn an arbitrary regex into a FSM later.)

Familiarising yourself with the FSM module and testfsm.c

- In the file **testfsm.c** you will see a static global variable called **myfsm**, initialized by a **designated initializer**:

```

static FSM myfsm = {    // FSM representation of our simple regex
    .nstates = 2,
    .successstate = 2,
    .state = {
        {
            // state 0 - expect: 'a' | 'b'
            .narcs = 3,
            .arc = {
                { 'a', 0 },
                { 'b', 1 },
                { DEFAULT_TOK, FAIL },
            },
        },
        {
            // state 1: seen 'a'* 'b'+, expect: 'b' | 'c' | 'd'
            .narcs = 4,
            .arc = {
                { 'b', 1 },
                { 'c', 2 },
                { 'd', 2 },
                { DEFAULT_TOK, FAIL },
            },
        },
    },
},

```

which shows that:

- State 0 (first item in the `state[]` array) has 3 arcs: to state 0 if the next char is 'a', or to state 1 if the next char is 'b', or to the **FAIL** state otherwise.
- State 1 (2nd item in `state[]` array) has 4 arcs: to state 1 if the next char is 'b', to state 2 if the next char is 'c' or 'd', or to the **FAIL** state otherwise.

Note that the **DEFAULT** arc from the diagram is written as **DEFAULT_TOK** here in the C code. In the file **token.h** you will find **#defines** for special tokens such as **DEFAULT_TOK** (and a few others) that correspond to diagram labels such as **DEFAULT**.

- In the file **fsm.h** you will find the FSM type, and helper types **FSM_arc** and **FSM_state**:

```
// a single arc
typedef struct
{
    int      tok;           // if we see this token then
    int      nextstate;     // enter this state next
} FSM_arc;

// a whole FSM:
typedef struct
{
    int      nstates;       // number of states
    FSM_state state[FSM_MAXSTATES]; // state[0..nstates-1]
    int      successtate;   // the success state
    // some other fields
} FSM;
```

```
// a single state
typedef struct
{
    int      narcs;         // number of arcs
    FSM_arc  arc[FSM_MAXARCS]; // arc[0..narcs-1]
} FSM_state;
```

Suppose we were to run this FSM given the target string "aabbbcwibble". Because our regex is anchored at the start, it turns out that we only need a single ASM to do this:

State	First Unused character	Action	Remaining target string
0	'a'	stay in state 0	"aabbbcwibble"
0	'a'	stay in state 0	"bbbcwibble"
0	'b'	enter state 1	"bbcwibble"
1	'b'	stay in state 1	"bcwibble"
1	'b'	stay in state 1	"cwibble"
1	'c'	enter state 2	"wibble"

As state 2 is the success state, the regex has now matched successfully against the string "aabbbc" (we don't care about the trailing "wibble").

Suppose instead we run this FSM given the target string "aba":

State	First Unused character	Action	Remaining target string
0	'a'	stay in state 0	"ba"
0	'b'	enter state 1	"a"
1	'a'	enter state FAIL	""

This last action happens because 'a' is not one of the state 1 arc labels – 'b', 'c' or 'd' – so we move into the **DEFAULT** state (**FAIL**), causing the FSM machinery to stop running and fail.

So the regex (correctly) fails to match the target string "aba" – there was no 'c' or 'd'.

Fleets of Active Finite State Machines (ASMs)

When do we need more than one ASM instance in our fleet? The obvious situation is when we attempt to match a regex that is **not anchored at the start**, but there are other situations that require more than one ASM too.

If we take our previous regex example, but remove the ^ start anchor – giving the regex **a*b+[cd]** – this will now match the *zero-or-more-'a's, followed by one-or-more-'b's, followed by 'c'-or-'d'* pattern starting at **any position in the target string**.

The **static FSM** is unchanged - exactly the same states and rules are used.

An ASM is represented in `fsm.h` as a structure containing a number, a current state and a private piece of data. When using FSMs to match regexes the **int data** field stores the **start pos in the target string of an attempted match**:

```
typedef struct
{
    int    number;    // this ASM's logical number
    int    currstate; // the current state
    int    data;      // this ASM's private data
} ASM;
```

We create a new ASM (in state 0) for every character in the target string. Each ASM represents the attempt to match the regex against the target string starting at a different position (the position stored in the **data** field). So at any one time we may have any number of ASMs in operation, and each time we move to the next character of the target string, all the ASMs receive that next target character in lockstep.

This is harder to display, but let's have a go. We are helped by the fact that many ASM instances start up, fail to match any of their non-default arc labels, and die immediately.

Let's see what happens when we attempt to match our regex against the target string "ababc". What do we expect to happen? We can use `labsearchstr` to check what should happen:

```
../labsearchstr 'a*b+[cd]' ababc
have matched RE /a*b+[cd]/ against target: ab<abc>
```

So we expect it to match "abc" starting at position 2 in the target string.

We start with a single ASM instance. Let's assume that ASM numbers start from 1 and increase monotonically as new ASMs are created:

After	Nasms	ASM Details			Notes
		Number	State	Data	
pos 0: 'a'	1	1	0	0	ASM 1 represents "attempt to match starting at pos 0 in target string" (pos 0 is stored in the data field) It matches "a" and stays in state 0
pos 1: 'b'	2	1	1	0	ASM 1 matches "b", and enters state 1 – having provisionally matched "ab" starting at position 0
		2	1	1	ASM 2 is created, representing "attempt to match starting at pos 1", matches "b", enters state 1
pos 2: 'a'	1	3	0	2	ASM 1 and 2 die (because "a" cannot follow "b") ASM 3 is created, representing "attempt to match starting at pos 2", matches "a", stays in state 0
pos 3: 'b'	2	3	1	2	ASM 3 matches "b", enters state 1 – having provisionally matched "ab" starting at position 2
		4	1	3	ASM 4 is created, representing "attempt to match starting at pos 3", matches "b", enters state 1
pos 4: 'c'	0				ASM 3 matches "c" and enters state 2 – the success state – meaning that it has successfully matched "abc" starting at pos 2. Then all ASMs are killed

The Fleet data structure in the FSM module

- Continuing reading **fsm.h**, you will see that the **fleet of ASMs** is represented as a dynamic array of ASMs:

```
struct FSM_fleet                                // a dynamic array of ASMs
{
    int        len;                             // current number of members
    int        cap;                             // capacity of dynarray
    ASM        *actsm;                          // pointer to block of cap ASMs
    FSM        *fsm;                            // all ASMs will run this fsm
    // some other fields
};

typedef struct FSM_fleet *FSM_fleet;    // FSM_fleet is the user-facing ADT
```

- When it comes to functions in the FSM module, the most important ones are as follows:

```
// FSM_fleet l = fsm_create_fleet( &fsm, &dc );
//      Create a new fleet of ASMs.
//      There can be any number of ASMs running at the same time,
//      each with the same FSM machine, but each having a different ASM no,
//      current state and a piece of private data (created by calling dc).
//      Initially, we create one ASM in state 0.

// fsm_addto_fleet( l, state, data );
//      create a new ASM in the given state and append it to the fleet.

// fsm_update_fleet( l, tok );
//      Update the fleet l, passing the next token tok to all ASMs.
//      This might cause any number of ASMs to fail (and be removed),
//      or each ASM change state, or one ASM to succeed and
//      call a final success action (causing all ASMs to be removed
//      from the fleet so we start again clean).
//      It may also clone a fresh ASM..

// fsm_destroy_fleet( l );
//      Destroy the whole fleet of ASMs.
```

- In case you're curious about how the current position in the target string ends up in the **data** field in each AST as described above, in Part 3 the **data creator** callback function delivers the current position in the target string via a global variable, that current position is then stored in the **data** field, but in fact here in Part 2, **testfsm.c** ignores the **data** field entirely and the **data creator** callback function returns a fixed integer value – so obviously **testfsm** here in Part 2 cannot test fleets of ASMs because the **data** field is not set correctly. It only tests our single anchored FSM.

Specific Tasks in Part 2

- The unit test program `./testfsm` (invoked directly or via `cb --test`) trips an assertion and fails immediately. Remember the advice from part 1 about running test programs directly if there's a chance they may abort.
- There are several incomplete functions in `fsm.c`. You must add code to the incomplete functions to make all the fsm unit tests work successfully. There will be some hints in the code.
- Specifically, task 2a involves finishing `fsm_create_fleet()` for 10%. The fleet is a dynamic array of ASM structures, and the “dynamic array creation” code is missing.
- Task 2b involves finishing `fsm_addto_fleet()` for 10%, again some crucial “dynamic array code” is missing.
- Task 2c involves finishing `find_matching_arcs()` for 10%. Note that there is always at least one matching arc, because every state has a default arc, which matches if none of the specific arcs do. There are many hints in the code.
- Task 2d involves implementing the last part of `fsm_update_fleet()` which removes all failed ASMs from the fleet – for 10%.
- Remember to use `valgrind` to check for memory leaks. I won't remind you again!

Part 3 : Turning a Regex into a FSM (50%)

You are supplied with a number of files in the `~/re/Part3` directory:

README:	readme file that describes Part 3.
makefsm.[ch]:	RE to FSM conversion module - skeleton.
testmakefsm.c:	Unit test program for makefsm module.
match.[ch]:	regex string matching and reporting code - complete.
testmatch.c:	Unit test program for regex matching.
searchstr.c:	Match a regex to a target string on the command line.
.cbuild:	Custom CB config file to compile unit tests.

Getting started with Part 3

1. Cd into the **Part3/** directory, read the **README** file, then compile everything up via `cb`. Run the unit test `./testmakefsm` directly and you will see that it fails a large number of tests, and then seg faults.

In `makefsm.c`, the broad structure of constructing a FSM from a RE has been implemented in 4 phases, however it is incomplete – leading to the seg fault we just observed.

2. Note that you are not using your `fsm.c` code from Part 2 here in Part 3. You are using the pre-compiled version (`fsm.o`) of our sample solution for `fsm.c`, stored in `../lib/libsupport.a`, along with the `fsm.h` in `../lib/fsm.h`, which should be identical to the version in Part2. This allows you to attempt Part 3 entirely separately from Part 2.
3. Recall that the FSM module defines types for states and arcs. When `makefsm.c` wants to add an arc to a particular state in the FSM it is building, it calls a function:

```
// arc( state, tok, nextstate );
//   Add an arc from state (an initialized FSM_state *) for tok to nextstate
```

That function is incomplete – you will need to complete it. Refer back to those types in Part 2 (or in `../lib/fsm.h`) to work out how to add an arc.

4. There is another function which adds arcs for every member of a set:

```
// arc_set( sn, set, nextstate );
//   add arc(sn, member, nextstate) for each member of set
```

This function has been implemented for you. It's a loop that calls `arc()`.

5. In phase 1 of making an FSM, each simple pattern in the RE is turned into a snippet of 1 or more new states (along with their arcs), which are appended to a list of states in the FSM we are building.
6. Note that phase 1 needs the concept of *unlabelled* arcs to represent the tricky problem of optionally matching something, or moving to a later state without consuming a character from the target string. Later phases remove these unlabelled arcs.

7. Most of the work of phase 1 is done by a helper function:

```
// FSM fsm;
// append_spat( spat, &fsm );
//     Append one or more states to fsm that implement
//     matching for SimplePat spat
```

8. Recall that a SimplePat is a pair (**SimpleType**, **OptStr**), corresponding broadly to the following Haskellish data structure:

```
data SimplePat = Dot or OptDot or DotStar or DotPlus or One(set) or Opt(set) or Star(set) or Plus(set)
```

(with the OptStr used to represent the set in the last 4 shapes, or set to NULL for the first four).

9. `append_spat()` is incomplete: 4 simple patterns (**Dot**, **One**, **OptDot** and **Opt**) have been implemented in a **switch** (see figure 1), but the other 4 simple patterns (all the **Star** and **Plus** repetition patterns) have not been implemented (see figure 2). Note that **DotPlus** and **Plus(set)** require an extra state, and there is already code above the switch to create it.

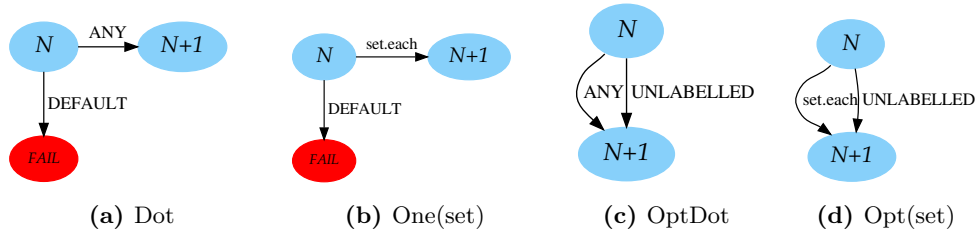


Figure 1: Simple Patterns already handled

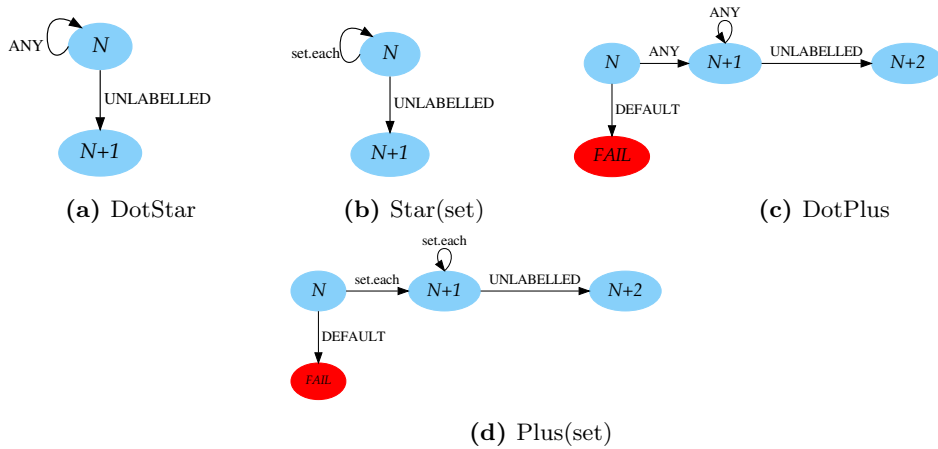


Figure 2: Simple Patterns not yet handled

10. Note that in those figures, single arcs are shown labelled with a single token (such as **ANY**, **DEFAULT** and **UNLABELLED**), which turn into calls to `arc()` – whereas sets of arcs are labelled with `set.each`, which turn into calls to `arc_set()`. If you compare the code written to handle `One(set)` and `Opt(set)` to figures 1(b) and 1(d), you should clearly see the `arc()` vs `arc_set()` distinction.
11. In phase 2 of making an FSM, we handle the optional end anchor, and also replace an optional final unlabelled arc with a default arc (if one is present). Figure 3 shows how the end anchor should be handled if present.

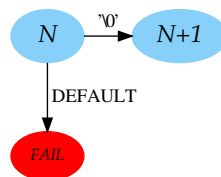


Figure 3: End Anchor

Here, the `'\0'` matches the end of the target string. In this case, state `N+1` will be the final state – the success state. So Figure 3 means that the FSM only succeeds if the target string has been exhausted by the earlier states and arcs. If any of the target string has not been consumed by earlier matching, then the `DEFAULT` rule kicks in and we fail.

12. In phase 3 of making an FSM, we handle the (rather complicated) removal of unlabelled arcs. Much of the complexity of this technique occurs in this phase – but **you will be relieved to know that we’ve implemented it all for you**.
13. Finally, in phase 4 of making an FSM, any unreachable states (created by phase 3) can be removed in a *renumber* operation – the unreachable states are removed, and the destination states in any arcs in later states must be adjusted.
14. This requires two pieces of work: first, identifying which states are reachable (i.e. have any arcs going into them) and recording this info in a `bool reachable[MAXSTATES]` array. Second, carefully deleting unreachable states. You will tackle the first piece of work – identifying reachable states.
15. The main RE to FSM conversion function:

```

// FSM fsm;
// regex_to_fsm( re, &fsm );
//      Convert an abstract regex re into a finite-state-machine fsm,
//      in 4 main phases. nb: can ignore startanchor throughout, because
//      the match infrastructure deals with this now.

```

calls all 4 phases sequentially and then tidies up. It has been written for you.

Specific Tasks in Part 3

1. In more detail, you are to:
 - Task 3a: Complete `arc()` for 10%. This should also make `arc_set()` work.
 - Task 3b: 4 cases are missing in `append_spat()`: Implement them for 20%, referring to figures 1 and 2 for guidance.
 - Task 3c: `phase2()`'s code to deal with an endanchor is missing, add it for 10% – referring to Figure 3 to see how the end anchor is handled when present.
 - Task 3d: implement the `compute_reachable()` function (part of phase 4) that computes a `reachable[state]` mapping, for 10%.
2. If you complete all these tasks, the test programs (run by `cb --test`) should all complete, and all succeed. If you can't get `compute_reachable()` to work, this will break some of the phase 4 tests. To confirm that is happening, comment out the call to `phase4()` in `regex_to_fsm()`, recompile and then see if all phase 1 to phase 3 tests work. If so, that strongly suggests that your `compute_reachable()` function is broken.

BONUS : Checking all your code works together – no marks

1. If you have a few minutes left at the end, and have got all of parts 1-3 working, you might like to try assembling all your solutions together and see if they give you a working regex pattern matching system – just for the satisfaction.
2. Cd into the `BONUS/` directory, and run `./copyin`. That should copy in your solutions from `../Part*`. Then you should be able to run `cb` to compile all your code together. If all goes well, all the unit tests and the final program `searchstr` should work.
3. Invoke `searchstr` just like the top-level README described running `labsearchstr`. In fact, why not run both `labsearchstr` and `searchstr` on each test regex and target string, and compare the results.

Good luck!